

SISTEMAS BASEADOS EM REGRAS

Aplicações Inteligentes no Dia a Dia

Disciplina: Inteligência Artificial

Professor: Claudinei Dias (Ney)

Semestre: 2026/1

1. Introdução

A Inteligência Artificial (IA) é um campo da Ciência da Computação dedicado a desenvolver sistemas capazes de executar tarefas que normalmente requerem inteligência humana, como raciocínio, aprendizado, reconhecimento de padrões e tomada de decisão. Dentro desse campo, existem duas grandes vertentes: a IA simbólica, baseada em representações explícitas do conhecimento e raciocínio lógico, e a IA subsimbólica, baseada em aprendizado estatístico a partir de dados — como redes neurais e aprendizado de máquina.

Os Sistemas Baseados em Regras (SBR), também conhecidos como Sistemas Especialistas ou Sistemas de Produção, pertencem à vertente simbólica e representam uma das abordagens mais consolidadas e historicamente relevantes da IA. Eles reproduzem o raciocínio de especialistas humanos por meio de regras condicionais do tipo SE \<condição\> ENTÃO \<ação\>, capturando formalmente o conhecimento de domínio de profissionais experientes — médicos, engenheiros, advogados, analistas financeiros — e permitindo que sistemas computacionais tomem decisões com o mesmo nível de especialização.

O desenvolvimento dos primeiros sistemas especialistas ocorreu nos anos 1960 e 1970, com projetos pioneiros como o DENDRAL (1965), que identificava compostos químicos orgânicos a partir de dados de espectrometria de massa, e o MYCIN (1972), que diagnosticava infecções bacterianas e recomendava antibióticos. Esses sistemas demonstraram, pela primeira vez, que computadores podiam igualar ou até superar o desempenho de especialistas humanos em domínios restritos e bem definidos. A partir daí, os SBR se proliferaram em indústrias como saúde, finanças, telecomunicações e manufatura.

Mesmo com o crescimento explosivo do aprendizado de máquina nas últimas décadas, os Sistemas Baseados em Regras continuam amplamente adotados em aplicações reais, especialmente em contextos onde a explicabilidade e a auditabilidade das decisões são requisitos obrigatórios. Regulamentações como o AI Act da União Europeia (2024) e normas setoriais de saúde e finanças impõem a obrigação de justificar decisões automatizadas — algo que os SBR fazem naturalmente, ao contrário de modelos de aprendizado de máquina do tipo "caixa-preta".

Este trabalho tem como objetivo apresentar os fundamentos teóricos dos Sistemas Baseados em Regras, suas principais técnicas, ferramentas e frameworks, exemplos de aplicação em casos reais, e uma implementação prática completa: um Sistema Especialista de Triagem Médica Básica desenvolvido em Python, disponibilizado em duas modalidades — interface de linha de comando e interface web interativa com Flask.

2. Fundamentos Teóricos

2.1 O que são Sistemas Baseados em Regras

Um Sistema Baseado em Regras é um programa de computador que utiliza um conjunto de regras lógicas para derivar conclusões a partir de fatos conhecidos sobre uma situação. Cada regra é uma sentença condicional na forma:

```
SE <antecedente> ENTÃO <consequente>
```

O antecedente (ou premissa, lado esquerdo — LHS, Left-Hand Side) descreve uma ou mais condições que devem ser verdadeiras para que a regra seja ativada. O consequente (ou conclusão, lado direito — RHS, Right-Hand Side) descreve a ação a ser executada ou o novo fato a ser inserido na base de fatos quando a regra é disparada.

Exemplos de regras de produção:

```
SE paciente_tem_febre = verdadeiro
  E paciente_tem_tosse = verdadeiro
  E paciente_perdeu_olfato = verdadeiro
ENTÃO diagnostico = "Suspeita de COVID-19"

SE score_credito > 700
  E renda_mensal > 5000
  E sem_dividas_ativas = verdadeiro
ENTÃO decisao_credito = "APROVADO"

SE temperatura_motor > 120
  E vibracao > limiar_normal
ENTÃO acionar_alerta_manutencao()
```

Os SBR pertencem ao paradigma da IA simbólica — o conhecimento é representado de forma explícita, declarativa e interpretável por humanos. Isso os diferencia fundamentalmente das redes neurais, onde o conhecimento está distribuído em milhões de parâmetros numéricos sem significado semântico direto.

2.2 Arquitetura de um Sistema Baseado em Regras

A arquitetura clássica de um Sistema Baseado em Regras é composta por três componentes essenciais, mais a interface com o usuário:

1. Base de Conhecimento (Knowledge Base)

Armazena o conhecimento especializado do domínio na forma de regras de produção. É construída através de um processo de aquisição de conhecimento (knowledge engineering), no qual um engenheiro de conhecimento entrevista especialistas humanos e traduz seu conhecimento tácito em regras formais. A qualidade e a abrangência da base de conhecimento determinam diretamente a qualidade das respostas do sistema.

2. Base de Fatos (Working Memory / Memória de Trabalho)

Armazena os fatos conhecidos sobre a situação atual sendo analisada. Esses fatos podem

- Fatos iniciais: inseridos diretamente pelo usuário (ex: "o paciente tem febre")
- Fatos derivados: gerados automaticamente pelo motor de inferência ao disparar regras (ex: "o paciente tem sintomas respiratórios" derivado de febre + tosse)

A base de fatos é dinâmica — é atualizada a cada ciclo do motor de inferência.

3. Motor de Inferência (Inference Engine)

É o núcleo computacional do sistema. Responsável por combinar as regras da base de conhecimento com os fatos da memória de trabalho para gerar novas conclusões. O motor executa continuamente um ciclo de três etapas:

Ciclo Reconhecimento-Ação (Recognize-Act Cycle):

1. CORRESPONDÊNCIA (Match)

Verifica quais regras têm todos os seus antecedentes satisfeitos pelos fatos atuais da memória de trabalho.
Forma o "conjunto conflito" (conflict set).

2. RESOLUÇÃO DE CONFLITOS (Conflict Resolution)

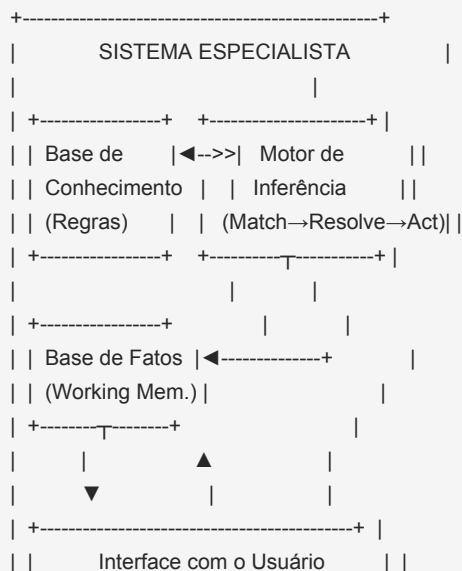
Quando múltiplas regras estão no conjunto conflito, aplica uma estratégia para escolher qual executar primeiro.

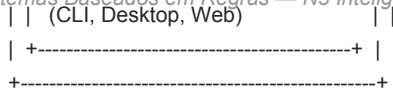
3. EXECUÇÃO (Act / Fire)

Executa o consequente da regra escolhida.
Atualiza a memória de trabalho com os novos fatos.
Retorna ao passo 1.

4. Interface com o Usuário

Componente responsável pela entrada de dados (coleta de fatos iniciais) e pela apresentação dos resultados e do raciocínio utilizado. Pode ser uma interface de linha de comando, uma aplicação desktop ou uma aplicação web.





2.3 Encadeamento para Frente (Forward Chaining)

O encadeamento para frente é uma estratégia de inferência orientada a dados (data-driven). O motor parte dos fatos iniciais disponíveis e aplica as regras de forma encadeada até atingir uma conclusão ou esgotar as possibilidades de inferência.

Algoritmo básico:

```

INÍCIO
  fatos ← fatos_iniciais_do_usuario
  ENQUANTO houver regras ativadas:
    conjunto_conflito ← {regras cujos antecedentes C= fatos}
    SE conjunto_conflito = vazio: PARAR
    regra_escolhida ← resolver_conflitos(conjunto_conflito)
    fatos ← fatos U {consequente da regra_escolhida}
  FIM ENQUANTO
  RETORNAR fatos
FIM
  
```

Exemplo de cadeia de inferência:

```

Passo 0 — Fatos iniciais:
  febre = verdadeiro
  tosse = verdadeiro
  perda_de_olfato = verdadeiro

Passo 1 — Conjunto conflito:
  R01: febre ^ tosse ^ dor_de_cabeca → Gripe [NÃO: dor_de_cabeca ausente]
  R02: febre ^ tosse ^ perda_de_olfato → COVID-19 [SIM: todas condições presentes]
  R03: febre ^ tosse ^ dif_respiratoria → Bactéria [NÃO: dif_respiratoria ausente]
  ...

Passo 2 — Resolução de conflitos:
  Apenas R02 está no conjunto conflito → R02 é disparada

Passo 3 — Execução:
  Novo fato: diagnostico = "Suspeita de COVID-19"

Passo 4 — Novo ciclo: nenhuma nova regra ativada → PARAR
  
```

O encadeamento para frente é a estratégia mais utilizada em sistemas de diagnóstico e monitoramento, pois parte das observações (sintomas, leituras de sensores, dados do cliente) e chega às conclusões.

2.4 Encadeamento para Trás (Backward Chaining)

O encadeamento para trás é uma estratégia orientada a objetivos (goal-driven). Em vez de partir dos fatos, parte de uma hipótese (objetivo) e tenta encontrar fatos que a comprovem.

```
PROVAR(hipótese):  
  SE hipótese in fatos_conhecidos: RETORNAR verdadeiro  
  regras_relevantes ← {regras cujo consequente = hipótese}  
  PARA cada regra em regras_relevantes:  
    SE PARA cada condição c em antecedente(regra): PROVAR(c)  
    ENTÃO RETORNAR verdadeiro  
  RETORNAR falso
```

Exemplo:

```
Objetivo: provar "paciente tem gripe"  
  
Regra R01: febre ^ tosse ^ dor_de_cabeca → gripe  
  
Sistema pergunta:  
  → O paciente tem febre? [usuário: sim]  
  → O paciente tem tosse? [usuário: sim]  
  → O paciente tem dor de cabeça? [usuário: sim]  
  
Todas as condições provadas → hipótese CONFIRMADA
```

O encadeamento para trás é a estratégia usada em linguagens de programação lógica como Prolog e em sistemas que trabalham com diagnósticos específicos pré-definidos.

2.5 Resolução de Conflitos

Quando múltiplas regras têm seus antecedentes satisfeitos simultaneamente (conjunto conflito não vazio), o motor precisa decidir qual executar. As principais estratégias são:

Estratégia	Descrição	Quando usar
Especificidade	Prioriza regras com mais condições (Diagnóstico médico, juridico	
Recência	Prioriza regras que usam os fatos ma Monitoramento em tempo real	
Ordem de declaração	Executa na ordem em que as regras	Sistemas simples
Prioridade explícita	Cada regra tem um peso definido ma	Sistemas críticos com hierarquia
Aleatoriedade	Escolhe aleatoriamente entre as canç	Simulações, jogos

Na implementação deste trabalho foi adotada a estratégia de especificidade: a regra com o maior número de condições tem prioridade, evitando que regras genéricas sobreponham diagnósticos mais precisos.

2.6 O Algoritmo Rete

Para grandes bases de conhecimento (centenas ou milhares de regras), verificar todas as regras a cada ciclo seria computacionalmente inviável. O algoritmo Rete (do latim rete, "rede") resolve esse problema de forma elegante.

O Rete compila as regras em uma rede de nós que representa os padrões de

correspondência. Em vez de re-avaliar todas as regras a cada ciclo, o Rete mantém uma estrutura de memória que armazena quais condições já foram satisfeitas. Quando um fato muda, apenas os nós afetados por aquele fato são re-avaliados.

O resultado é que a complexidade de correspondência deixa de ser proporcional ao número de regras e passa a ser proporcional ao número de mudanças nos fatos — tornando o sistema muito mais eficiente. O Rete é implementado pelo CLIPS, Drools e Experta.

2.7 Tratamento de Incerteza — Fatores de Certeza

Sistemas especialistas em domínios como medicina nem sempre podem trabalhar com certezas absolutas. Para lidar com isso, o MYCIN introduziu o conceito de fatores de certeza (Certainty Factors — CF), valores numéricos entre -1 e 1 que expressam o grau de confiança em um fato ou conclusão:

- CF = 1,0 → certeza absoluta (positiva)
- CF = 0,0 → incerto
- CF = -1,0 → certeza de que é falso

Regras com fatores de certeza têm a forma:

SE febre (CF=0.8) E tosse (CF=0.9)
ENTÃO gripe (CF = 0.72) ← produto dos CFs individuais

Esse mecanismo permite que o sistema trabalhe com informações parciais e incertas, retornando não apenas um diagnóstico mas também um grau de confiança nele.

2.8 Comparação com Aprendizado de Máquina

Para contextualizar a relevância dos SBR no cenário atual, a tabela abaixo compara as duas abordagens:

Característica	Sistemas Baseados em Regras	Aprendizado de Máquina
Representação	Regras explícitas (declarativas)	Parâmetros numéricos (implícita)
Explicabilidade	Alta — cada decisão é rastreável	Baixa — "caixa-preta"
Dados necessários	Pouco (especialista humano)	Muito (grandes datasets)
Atualização	Manual (editar regras)	Re-treinamento do modelo
Adaptabilidade	Baixa	Alta
Domínios ideais	Regras bem definidas, críticos	Padrões complexos, imagens, texto
Custo computacional	Baixo	Alto (treinamento)
Conformidade regulatória	Fácil de auditar	Difícil de justificar

3. Técnicas, Ferramentas e Frameworks

3.1 CLIPS (C Language Integrated Production System)

Desenvolvido pelo Software Technology Branch da NASA no Johnson Space Center em 1986, o CLIPS tornou-se a referência histórica dos sistemas especialistas de código aberto. Escrito em C para portabilidade máxima, implementa o algoritmo Rete e suporta programação orientada a objetos (COOL — CLIPS Object-Oriented Language).

Sintaxe de exemplo — Sistema de suporte à decisão de lançamento:

```
(defrule verificar-condicoes-tempo
  (vento-velocidade ?v &(< ?v 25))
  (chuva FALSE)
  (visibilidade-km ?vis &(> ?vis 5))
  =>
  (assert (condicoes-meteorologicas OK))
  (printout t "Condições meteorológicas aprovadas." crlf))

(defrule autorizar-lancamento
  (condicoes-meteorologicas OK)
  (combustivel-nivel ADEQUADO)
  (sistemas-checados TRUE)
  =>
  (assert (lancamento AUTORIZADO))
  (printout t "LANÇAMENTO AUTORIZADO." crlf))
```

O CLIPS ainda é usado em projetos de pesquisa, sistemas embarcados industriais e ensino de IA. A NASA continua o mantendo ativamente.

3.2 Drools (Red Hat / JBoss)

O Drools é o principal framework de automação de decisões no ecossistema Java empresarial. Mantido pela Red Hat, usa a linguagem DRL (Drools Rule Language) e implementa uma versão otimizada do Rete chamada Phreak. Integra-se nativamente com o ecossistema Spring/Java EE e fornece ferramentas gráficas para que analistas de negócios editem regras sem escrever código.

Sintaxe DRL — Sistema de concessão de crédito:

```
package br.com.banco.credito;

rule "Aprovação automática de crédito premium"
  salience 100
  when
    $sol : Solicitante(
      score >= 800,
      rendaMensal >= 10000.0,
      tempoEmprego >= 24,
      possuiRestricao == false
    )
  then
    $sol.setStatus("APROVADO_AUTOMATICO");
    $sol.setLimite($sol.getRendaMensal() * 5);
    insert(new Aprovacao($sol, "Perfil premium"));
```



```

        System.out.println("Crédito aprovado automaticamente: " + $sol.getNome());
    end

    rule "Encaminhar para análise manual"
        salience 50
        when
            $sol : Solicitante(
                score >= 600,
                score < 800,
                possuiRestricao == false
            )
        then
            $sol.setStatus("ANALISE_MANUAL");
            insert(new Tarefa($sol, "REVISAR_CREDITO"));
        end
    end

```

O Drools é adotado por instituições financeiras como Itaú, Bradesco e grandes seguradoras para processar milhões de decisões de crédito por dia com baixíssima latência.

3.3 Experta (Python)

O Experta é um port moderno do CLIPS para Python 3, criado para tornar o desenvolvimento de sistemas especialistas acessível no ecossistema Python. Usa decoradores para definir regras de forma pythônica e implementa o algoritmo Rete para eficiência.

Exemplo completo com Experta:

```

from experta import *

class Sintoma(Fact):
    """Representa um sintoma do paciente."""
    pass

class Diagnostico(Fact):
    """Representa uma conclusão diagnóstica."""
    pass

class SistemaTriagem(KnowledgeEngine):

    @Rule(Sintoma(nome='febre'),
          Sintoma(nome='tosse'),
          Sintoma(nome='perda_olfato'))
    def covid_suspeito(self):
        self.declare(Diagnostico(
            resultado='COVID-19 (suspeita)',
            orientacao='Isolamento e avaliação médica urgente'
        ))

    @Rule(Sintoma(nome='febre'),
          Sintoma(nome='tosse'),
          Sintoma(nome='dor_cabeca'))

```

```
def gripe_comum(self):
    self.declare(Diagnostico(
        resultado='Gripe comum',
        orientacao='Repouso, hidratação e antitérmico'
    ))

engine = SistemaTriagem()
engine.reset()
engine.declare(Sintoma(nome='febre'))
engine.declare(Sintoma(nome='tosse'))
engine.declare(Sintoma(nome='perda_olfato'))
engine.run()
```

3.4 PyKE (Python Knowledge Engine)

O PyKE é uma biblioteca Python acadêmica que implementa tanto encadeamento para frente quanto para trás, com uma sintaxe inspirada no Prolog. Suporta regras com variáveis e padrões complexos, sendo útil para pesquisa e demonstrações de raciocínio lógico dedutivo.

3.5 OpenL Tablets

Uma solução empresarial baseada em planilhas Excel como interface de edição de regras. Analistas de negócio definem as regras em formato tabular no Excel, e o OpenL Tablets as compila para código Java em tempo de execução. Muito usado no setor de seguros europeu.

3.6 Comparativo Completo

Framework	Linguagem	Algoritmo	Licença	Uso Principal
CLIPS	CLIPS	Rete	LGPL	Pesquisa / NASA / Legado
Drools	Java/DRL	Phreak	Apache 2.0	Mercado empresarial Java
Experta	Python	Rete	Apache 2.0	Prototipagem rápida
PyKE	Python	F/B Chaining	MIT	Acadêmico
OpenL Tablets	Java/Excel	Próprio	LGPL	Seguros / Finanças
Impl. própria	Python	Linear	—	Educacional

4. Exemplos Reais e Estudos de Caso

4.1 MYCIN — Diagnóstico de Infecções Bacterianas (Stanford, 1972)

Desenvolvido por Edward Shortliffe na Universidade de Stanford entre 1972 e 1976, o MYCIN é considerado o sistema especialista médico mais influente da história. Seu objetivo era auxiliar médicos no diagnóstico de infecções bacterianas no sangue e meningite, e na recomendação de antibióticos adequados.

Características técnicas:

- Base de conhecimento com cerca de 600 regras de produção
- Motor de inferência por encadeamento para trás
- Uso pioneiro de fatores de certeza para lidar com incerteza diagnóstica
- Interface em linguagem natural (inglês) para interação com médicos

Resultados:

Em um estudo comparativo, o MYCIN foi avaliado por um painel de especialistas em doenças infecciosas que não sabiam se as recomendações vinham do sistema ou de médicos humanos. O MYCIN obteve 65% de aprovação — igual ou superior à média dos médicos residentes e internos testados no mesmo conjunto de casos. Apesar do sucesso técnico, o sistema nunca foi implantado em hospitais por questões regulatórias e de responsabilidade médica — mas seu impacto foi imenso: o MYCIN influenciou diretamente todos os sistemas especialistas médicos subsequentes e o design dos frameworks modernos.

4.2 XCON / R1 — Configuração de Computadores (DEC, 1980)

Desenvolvido pela Digital Equipment Corporation (DEC) em parceria com a Carnegie Mellon University, o XCON (eXpert CONfigurer), também chamado R1, automatizava a configuração de pedidos de computadores VAX.

O problema: Cada pedido de computador VAX envolvia centenas de componentes (processadores, memórias, cabos, gabinetes, placas) que precisavam ser selecionados e combinados corretamente. Erros de configuração eram frequentes e custosos.

A solução: O XCON analisava cada pedido e determinava automaticamente quais componentes eram necessários, em que quantidade e como organizá-los nos gabinetes.

Resultados:

- Base de conhecimento com mais de 2.500 regras
- Taxa de acerto acima de 98% (vs. ~70% dos técnicos humanos)
- Economia estimada de US\$ 25 milhões por ano em custos de retrabalho
- Reduziu o tempo de processamento de pedidos de semanas para horas

O XCON foi o primeiro sistema especialista a demonstrar retorno financeiro mensurável em escala industrial, detonando o boom dos sistemas especialistas nos anos 1980.

4.3 Sistemas de Aprovação de Crédito (Setor Financeiro)

O setor financeiro é um dos maiores usuários de Sistemas Baseados em Regras no mundo. Praticamente todos os grandes bancos brasileiros — Itaú, Bradesco, Santander, Banco do Brasil, Caixa — utilizam SBR para decisões de crédito.

Exemplo de regras típicas de um sistema de crédito:

```
SE score_serasa >= 800
  E renda_comprovada >= R$ 5.000
  E tempo_emprego >= 12 meses
```

```
E sem_restricoes_cpf = verdadeiro
ENTÃO aprovação_automática(limite = renda * 3)

SE score_serasa >= 600 E score_serasa < 800
  E renda_comprovada >= R$ 3.000
  ENTÃO encaminhar_analise_humana()

SE restricao_spc = verdadeiro
  OU negativacao_serasa = verdadeiro
  ENTÃO recusar_credito(motivo = "Restrições cadastrais")
```

Por que SBR e não ML?

Decisões de crédito precisam ser explicadas ao cliente quando negadas (exigência do Banco Central). Com SBR, o sistema pode informar exatamente qual regra foi ativada: "seu pedido foi negado porque seu score (580) está abaixo do mínimo exigido (600)". Com modelos de ML, essa explicação é muito mais difícil.

O Drools é o framework dominante nesse segmento, processando milhões de análises de crédito por dia com latência inferior a 100ms.

4.4 Diagnóstico de Falhas em Telecomunicações

Empresas de telecomunicações como Vivo (Telefônica Brasil), Claro e TIM utilizam SBR para triagem automática de chamados técnicos antes de escalá-los a equipes humanas.

Fluxo típico:

```
Cliente relata: "Internet caiu"
↓
SBR executa diagnóstico automático:
  SE sinal_onu = "sem luz" → cabo de fibra cortado
  SE onu_piscando_vermelho → problema na OLT da central
  SE sinal_ok E velocidade_baixa → congestionamento
  SE todos_vizinhos_afetados → falha na rede externa
↓
Sistema gera OS automática com diagnóstico
↓
Técnico recebe OS já com causa provável identificada
```

Resultados mensuráveis:

- Redução de 35–40% no Tempo Médio de Atendimento (TMA)
- Eliminação de 60% das visitas técnicas desnecessárias ("visita branca")
- Melhoria no NPS (Net Promoter Score) dos clientes

4.5 Manutenção Preditiva na Aviação (Embraer)

A Embraer integra sistemas baseados em regras com dados de telemetria de suas aeronaves para alertas de manutenção preditiva. Sensores distribuídos pela aeronave coletam dados continuamente, e um SBR analisa esses dados em tempo real:

SE temperatura_motor_1 > 850°C

E temperatura_motor_1 > temperatura_motor_2 + 50°C

ENTÃO alertar("Assimetria térmica no motor 1 — verificar combustão")

SE vibração_trem_principal > 3.2g

E vibração_trem_principal AUMENTANDO por 3 ciclos consecutivos

ENTÃO alertar("Vibração anômala no trem de pouso — inspeção obrigatória")

Esse sistema evita que falhas mecânicas progressivas se tornem incidentes, ao mesmo tempo em que otimiza os intervalos de manutenção — reduzindo paradas não programadas em até 25%.

4.6 Sistemas de Combate a Fraudes

Bancos e fintechs utilizam SBR como primeira linha de defesa contra fraudes em transações em tempo real:

```
SE pais_transacao ≠ pais_habitual_do_cliente
```

```
E valor > R$ 500
```

```
E horario in [00:00, 06:00]
```

```
ENTÃO bloquear_transacao(); notificar_cliente()
```

```
SE N_transacoes_1h > 10
```

```
E N_transacoes_dia > 20
```

```
ENTÃO marcar_para_analise_fraude()
```

As regras permitem reação imediata (em menos de 50ms, antes da transação ser processada) e são facilmente ajustáveis pelos analistas de fraude sem necessidade de re-treinar modelos.

5. Proposta de Aplicação

5.1 Visão Geral

Para demonstrar os conceitos estudados, foi desenvolvido um Sistema Especialista de Triagem Médica Básica em Python. O sistema analisa os sintomas informados pelo usuário e sugere um diagnóstico provável com orientação básica, exibindo com transparência quais regras foram ativadas e qual foi o raciocínio seguido.

O domínio de triagem médica foi escolhido por três razões:

- É o domínio clássico dos sistemas especialistas, com referências históricas sólidas (MYCIN)
- As regras são intuitivas para qualquer pessoa, facilitando a compreensão do mecanismo
- Tem aplicação direta em contextos reais: triagem em UBSs, teleatendimento médico, aplicativos de saúde

O sistema foi disponibilizado em duas modalidades:

- Interface de Linha de Comando (CLI): para execução direta no terminal, sem dependências
- Interface Web (Flask): para demonstração visual interativa no navegador

5.2 Base de Conhecimento

A base de conhecimento contém 10 regras de produção e 16 fatos (sintomas possíveis):

Regra	Diagnóstico	Condições (antecedente)
R01	Gripe comum	febre + tosse + dor de cabeça
R02	Suspeita COVID-19	febre + tosse + perda de olfato
R03	Infecção bacteriana respiratória	febre + tosse + dificuldade respiratória
R04	Enxaqueca	dor de cabeça + náusea + sensibilidade à luz
R05	Infecção intestinal	náusea + vômito + diarreia
R06	Alergia respiratória	espirros + coriza + olhos vermelhos
R07	Resfriado comum	espirros + coriza + tosse
R08	Hipoglicemia	tontura + suor frio + tremores
R09	Suspeita de hipertensão	dor de cabeça + tontura + visão turva
R10	Fallback	(nenhuma regra satisfeita)

Representação em Python:

```
REGRAS = [
    {
        "id": "R01",
        "descricao": "Gripe comum",
        "condicoes": {"febre", "tosse", "dor_de_cabeca"},
        "conclusao": "Gripe comum — repouso, hidratação e antitérmico se necessário.",
    },
    {
        "id": "R02",
        "descricao": "COVID-19 (suspeita)",
        "condicoes": {"febre", "tosse", "perda_de_olfato"},
        "conclusao": "Suspeita de COVID-19 — procure um médico e faça isolamento preventivo.",
    },
    # ... demais regras
]
```

A escolha de conjuntos Python (sets) para representar as condições permite verificar de forma eficiente se todas as condições de uma regra estão presentes nos fatos usando a operação `issubset()`.

5.3 Motor de Inferência por Encadeamento para Frente

O motor de inferência percorre todas as regras e verifica quais têm suas condições completamente satisfeitas pelos fatos informados. Em seguida, aplica resolução de conflitos por especificidade:

```
def encadeamento_para_frente(fatos):
    disparadas = []
    for regra in REGRAS:
        # Verificação: todas as condições da regra estão nos fatos?
        if regra["condicoes"] and regra["condicoes"].issubset(fatos):
```

```
# Resolução de conflitos: mais condições = mais específica = maior prioridade
disparadas.sort(key=lambda r: len(r["condicoes"]), reverse=True)
return disparadas
```

Raciocínio por exemplo:

Suponha que o usuário informe: febre, tosse e dor de cabeça.

```
Fatos = {febre, tosse, dor_de_cabeca}

R01: {febre, tosse, dor_de_cabeca} C= {febre, tosse, dor_de_cabeca} → VERDADEIRO [ok]
R02: {febre, tosse, perda_de_olfato} C= {febre, tosse, dor_de_cabeca} → FALSO [x]
R07: {espirros, coriza, tosse} C= {febre, tosse, dor_de_cabeca} → FALSO [x]
...

Conjunto conflito: [R01]
Regra disparada: R01
Conclusão: "Gripe comum"
```

5.4 Interface de Linha de Comando (CLI)

A versão CLI coleta os sintomas via terminal com perguntas de resposta sim/não e exibe o resultado com formatação textual:

```
SISTEMA ESPECIALISTA DE TRIAGEM MÉDICA
-----
Responda as perguntas abaixo com 's' (sim) ou 'n' (não).

Você tem febre? [s/n]: s
Você está com tosse? [s/n]: s
Você tem dor de cabeça? [s/n]: s
...

=====
RESULTADO DA ANÁLISE
=====

>> Sintomas informados:
  • dor de cabeça
  • febre
  • tosse

>> Regras disparadas:
  [R01] Gripe comum — condições: dor de cabeça, febre, tosse

>> Conclusão:
  DIAGNÓSTICO: Gripe comum — repouso, hidratação e antitérmico se necessário.
```

Não requer instalação de dependências — funciona com Python 3.7+ padrão.

5.5 Interface Web com Flask

Para tornar a demonstração mais visual e acessível, foi desenvolvida uma interface web utilizando o microframework Flask do Python. A interface permite que o usuário marque os sintomas como checkboxes em uma página HTML e receba o diagnóstico de forma imediata, sem recarregar a página.

5.5.1 Tecnologias Utilizadas

Componente	Tecnologia	Justificativa
Servidor web	Flask 3.x	Leve, simples, padrão Python
Templates	Jinja2 (incluso no Flask)	Renderização HTML dinâmica
Frontend	HTML5 + CSS3 puro	Sem dependências externas
Comunicação	HTTP POST (formulário)	Simples e confiável

5.5.2 Arquitetura da Aplicação Web



5.5.3 Código do Servidor (app.py)

```

from flask import Flask, render_template, request
from sistema_especialista import REGRAS, TODOS_SINTOMAS, encadeamento_para_frente

app = Flask(__name__)

@app.route("/", methods=["GET", "POST"])
def index():
    resultado = None

    if request.method == "POST":
        # Coleta os fatos a partir dos checkboxes marcados
        fatos = {s for s in TODOS_SINTOMAS if request.form.get(s)}
        regras_disparadas = encadeamento_para_frente(fatos)

        if regras_disparadas:
            principal = regras_disparadas[0]
            alternativas = regras_disparadas[1:]
            conclusao = principal["conclusao"]
  
```



```

else:
    fallback = next(r for r in REGRAS if not r["condicoes"])
    conclusao = fallback["conclusao"]
    principal, alternativas = None, []

    resultado = {
        "fatos": sorted(fatos),
        "regras_disparadas": regras_disparadas,
        "principal": principal,
        "alternativas": alternativas,
        "conclusao": conclusao,
    }

    return render_template("index.html", sintomas=TODOES_SINTOMAS, resultado=resultado)

if __name__ == "__main__":
    app.run(debug=True)

```

O servidor é intencionalmente simples: uma única rota que aceita GET (para exibir o formulário) e POST (para processar os sintomas). O motor de inferência (encadeamento_para_frente) é reutilizado diretamente do módulo CLI sem modificação.

5.5.4 Interface Visual

A interface foi desenvolvida com tema escuro (dark mode) e design responsivo em CSS puro, sem frameworks externos como Bootstrap. Os sintomas são apresentados como cards de checkbox interativos — ao clicar, o card muda de cor indicando seleção. O resultado é exibido na mesma página abaixo do formulário, com três blocos distintos:

- Diagnóstico Principal — painel em destaque com o resultado mais provável
- Regras Disparadas — lista técnica mostrando quais regras foram ativadas e suas condições
- Diagnósticos Alternativos — outros diagnósticos possíveis com menor especificidade

Essa transparência na exibição do raciocínio é uma das vantagens fundamentais dos SBR: o usuário pode ver exatamente como o sistema chegou à conclusão.

5.5.5 Como Executar a Interface Web

```

# Instalar dependência
pip install flask

# Executar o servidor
python app.py

# Acessar no navegador
http://127.0.0.1:5000

```

5.5.6 Captura de Tela do Fluxo

```

+-----+
| Sistema Especialista de Triagem Médica |
| Sistemas Baseados em Regras — Inteligência Artificial |

```

[Forward Chaining]

```

+-----+
| Seleccione os sintomas presentes |
|
| [[ok]] Febre      [ ] Náusea    [ ] Espirros  |
| [[ok]] Tosse      [ ] Vômito    [ ] Coriza    |
| [[ok]] Dor de cabeça [ ] Diarreia [ ] Olhos verm. |
| [ ] Perda olfato [ ] Sens. luz [ ] Tontura    |
| [ ] Dif. respir. [ ] Suor frio  [ ] Tremores  |
| [ ] Visão turva   |
|
| [ ANALISAR SINTOMAS ] |
+-----+
| RESULTADO DA ANÁLISE |
|
| Diagnóstico Principal: |
| Gripe comum — repouso, hidratação e antitérmico |
|
| Sintomas informados: dor de cabeça · febre · tosse |
|
| Regras disparadas: |
| [R01] Gripe comum |
| Condições: dor de cabeça + febre + tosse |
|
| [!] Este sistema não substitui avaliação médica |
+-----+

```

5.6 Estrutura de Arquivos do Projeto

```

n3-ia/demo/
+-- sistema_especialista.py ← motor de regras + CLI
+-- app.py                  ← servidor Flask (interface web)
+-- requirements.txt        ← flask>=3.0
+-- README.md
+-- templates/
    +-- index.html          ← template HTML da interface web

```

5.7 Possibilidades de Extensão

O sistema implementado serve como ponto de partida para diversas extensões:

- Fatores de certeza: adicionar pesos às regras para lidar com incerteza, retornando porcentagens de confiança no diagnóstico
- Base de conhecimento maior: integrar com uma API médica (ex: ICD-10) para cobrir centenas de condições
- Persistência de histórico: salvar as consultas em banco de dados SQLite para análise de padrões
- Exportação de laudo: gerar PDF com o resultado da análise usando a biblioteca ReportLab
- Internacionalização: suporte a múltiplos idiomas com Flask-Babel

6. Conclusão

Os Sistemas Baseados em Regras representam um dos pilares mais sólidos e duradouros da Inteligência Artificial. Surgidos nos anos 1960 e consolidados na década de 1980 com sistemas como MYCIN e XCON, eles demonstraram que era possível capturar e automatizar o conhecimento de especialistas humanos com resultados concretos e mensuráveis.

A principal força dos SBR está na sua explicabilidade: ao contrário dos modelos de aprendizado de máquina que funcionam como "caixas-pretas", os sistemas baseados em regras podem justificar cada decisão com precisão, apontando exatamente qual regra foi ativada e por quê. Em 2024, o AI Act europeu tornou obrigatória a explicabilidade de sistemas de IA de alto risco — tornando os SBR não apenas uma escolha técnica, mas frequentemente uma exigência legal.

A comparação com o aprendizado de máquina revela que as duas abordagens são complementares, não concorrentes. Enquanto o ML é superior na detecção de padrões complexos em grandes volumes de dados não estruturados (imagens, texto, séries temporais), os SBR são insubstituíveis em domínios onde as regras são bem definidas, onde a conformidade regulatória exige auditabilidade, e onde especialistas humanos precisam ser capazes de entender e modificar o comportamento do sistema.

A implementação desenvolvida neste trabalho demonstrou esses princípios na prática: com Python puro e menos de 200 linhas de código (na versão CLI), foi possível construir um sistema especialista funcional capaz de simular o raciocínio de triagem médica. A adição de uma interface web com Flask ampliou a acessibilidade da demonstração sem modificar a lógica central do motor de inferência — evidenciando a separação clara entre raciocínio e apresentação, característica dos bons projetos de SBR.

Em suma, os Sistemas Baseados em Regras continuam mais relevantes do que nunca em 2026, tanto como solução autônoma em domínios bem estruturados quanto como componente de sistemas híbridos que combinam regras explícitas com aprendizado de máquina para obter o melhor dos dois mundos: adaptabilidade e explicabilidade.

7. Referências

RUSSELL, S.; NORVIG, P. *Inteligência Artificial: Uma Abordagem Moderna*. 4. ed. São Paulo: GEN LTC, 2022.

JACKSON, P. *Introduction to Expert Systems*. 3. ed. Harlow: Addison-Wesley, 1999.

GIARRATANO, J. C.; RILEY, G. *Expert Systems: Principles and Programming*. 4. ed. Boston: Course Technology, 2004.

WATERMAN, D. A. *A Guide to Expert Systems*. Reading: Addison-Wesley, 1986.

SHORTLIFFE, E. H. *Computer-Based Medical Consultations: MYCIN*. New York: Elsevier, 1976.

McDERMOTT, J. R1: A Rule-Based Configurer of Computer Systems. *Artificial Intelligence*,

FORGY, C. L. Rete: A Fast Algorithm for the Many Pattern/Many Object Pattern Match Problem. *Artificial Intelligence*, v. 19, n. 1, p. 17–37, 1982.

NASA. CLIPS Reference Manual. Versão 6.4. Johnson Space Center, 2021. Disponível em: clipsrules.net. Acesso em: jun. 2026.

RED HAT. Drools Documentation. Versão 8.x. Disponível em: drools.org. Acesso em: jun. 2026.

EXPERTA PROJECT. Experta: Expert System Shell for Python. Disponível em: experta.readthedocs.io. Acesso em: jun. 2026.

PALMIRANI, M. et al. Hybrid AI Architecture for Legal Reasoning: Rules and Machine Learning. In: JURIX 2021. Disponível em: frontiersin.org. Acesso em: jun. 2026.

EUROPEAN PARLIAMENT. Artificial Intelligence Act (AI Act). Regulation (EU) 2024/1689. Bruxelas: Parlamento Europeu, 2024.